

SQL Scenario-Based Interview Questions & Answers





Q: Write a query to find the second highest salary in the `employees` table.

```
SELECT MAX(salary) AS second_highest_salary
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

Q: Write a query to calculate the running total of sales for each day in the `sales` table.

```
SELECT sale_date, sale_amount,
       SUM(sale_amount) OVER (ORDER BY sale_date) AS running_total
FROM sales;
```

Q: Find the number of days between the order date and shipping date for each order.

```
SELECT order_id,  
       DATEDIFF(day, order_date, shipping_date) AS days_to_ship  
FROM orders;
```

Q: Retrieve the top 2 most expensive products per category from the `products` table.

```
SELECT category_id, product_name, price
FROM (
    SELECT category_id, product_name, price,
           ROW_NUMBER() OVER (PARTITION BY category_id ORDER BY price
DESC) AS rank
    FROM products
```

```
) AS ranked  
WHERE rank <= 2;
```

5. Using EXISTS with Subquery

Q: Find all customers who have placed at least one order in the last year.

A:

```
SELECT customer_id, customer_name  
FROM customers  
WHERE EXISTS (  
    SELECT 1 FROM orders  
    WHERE customers.customer_id = orders.customer_id  
    AND order_date >= DATEADD(year, -1, GETDATE())  
);
```

6. Cumulative Sum with Conditions

Q: Calculate a cumulative total of sales, but reset the total when a sale is above \$5000.

A:

```
SELECT sale_id, sale_amount,  
    SUM(sale_amount) OVER (PARTITION BY CASE WHEN sale_amount >  
5000 THEN sale_id ELSE 0 END ORDER BY sale_id) AS cumulative_total  
FROM sales;
```

7. Concatenate Multiple Rows

Q: Write a query to concatenate all product names in a category into a single string.

A:

```
SELECT category_id,  
    STRING_AGG(product_name, ', ') AS product_list  
FROM products  
GROUP BY category_id;
```

8. Avoiding Duplicate Entries

Q: Write a query to insert a new product only if it does not already exist in the `products` table.

A:

```
INSERT INTO products (product_name, price, category_id)  
SELECT 'New Product', 100, 1
```

```
WHERE NOT EXISTS (  
    SELECT 1 FROM products WHERE product_name = 'New Product'  
);
```

9. Index Usage for Optimization

Q: How would you improve the performance of a query that retrieves orders by customer ID?

A: Create an index on the `customer_id` column in the `orders` table.

```
CREATE INDEX idx_customer_id ON orders (customer_id);
```

10. Identify Records with Consecutive Values

Q: Write a query to identify employees with consecutive salary increases.

A:

```
SELECT employee_id, salary,  
       LAG(salary) OVER (PARTITION BY employee_id ORDER BY  
salary_date) AS previous_salary  
FROM employee_salaries  
WHERE salary > LAG(salary) OVER (PARTITION BY employee_id ORDER BY  
salary_date);
```

11. Data Pivoting

Q: Convert rows of monthly sales data into columns for each month.

A:

```
SELECT customer_id,  
       SUM(CASE WHEN MONTH(sale_date) = 1 THEN amount ELSE 0 END) AS  
Jan,  
       SUM(CASE WHEN MONTH(sale_date) = 2 THEN amount ELSE 0 END) AS  
Feb,  
       ...  
       SUM(CASE WHEN MONTH(sale_date) = 12 THEN amount ELSE 0 END) AS  
Dec  
FROM sales  
GROUP BY customer_id;
```

12. Check for Referential Integrity

Q: Write a query to find orders with a `customer_id` that does not exist in the `customers` table.

A:

```
SELECT order_id  
FROM orders  
WHERE customer_id NOT IN (SELECT customer_id FROM customers);
```

13. Finding Non-Matching Records with LEFT JOIN

Q: Find all customers who have never placed an order.

A:

```
SELECT c.customer_id, c.customer_name  
FROM customers c  
LEFT JOIN orders o ON c.customer_id = o.customer_id  
WHERE o.order_id IS NULL;
```

14. Conditional Aggregation

Q: Find the total sales amount, breaking down by paid and unpaid orders.

A:

```
SELECT  
    SUM(CASE WHEN payment_status = 'Paid' THEN amount ELSE 0 END) AS  
total_paid,  
    SUM(CASE WHEN payment_status = 'Unpaid' THEN amount ELSE 0 END)  
AS total_unpaid  
FROM orders;
```

15. Ranking Without Ties (Dense Rank)

Q: Retrieve employee details with a unique rank based on their salaries, without any ties.

A:

```
SELECT employee_id, name, salary,  
    DENSE_RANK() OVER (ORDER BY salary DESC) AS salary_rank  
FROM employees;
```

16. Cross Join for Combinations

Q: Generate a list of all product and vendor combinations.

A:

```
SELECT p.product_name, v.vendor_name  
FROM products p  
CROSS JOIN vendors v;
```

17. Using COALESCE for Null Handling

Q: Select the product name and description, replacing any NULL descriptions with “No Description Available.”

A:

```
SELECT product_name,  
       COALESCE(description, 'No Description Available') AS  
description  
FROM products;
```

18. Detecting Data Gaps

Q: Identify dates within the `orders` table where no orders were placed.

A:

```
SELECT DATEADD(day, n, (SELECT MIN(order_date) FROM orders)) AS  
missing_date  
FROM numbers  
WHERE DATEADD(day, n, (SELECT MIN(order_date) FROM orders)) NOT IN  
(SELECT order_date FROM orders);
```

19. Ensuring Unique Constraint Before Update

Q: Update an email address only if it does not duplicate an existing one.

A:

```
UPDATE employees  
SET email = 'new_email@example.com'  
WHERE employee_id = 123  
AND 'new_email@example.com' NOT IN (SELECT email FROM employees);
```

20. Rolling Average Calculation

Q: Calculate a rolling 7-day average of sales amounts.

A:

```
SELECT sale_date, sale_amount,  
       AVG(sale_amount) OVER (ORDER BY sale_date ROWS BETWEEN 6  
PRECEDING AND CURRENT ROW) AS rolling_7_day_avg  
FROM sales;
```
